REMORA: Real-time Expressive Motion to Output Routing Audio

Timothée Dravet
Polytech Marseille, Aix-Marseille Université
France
timothee.dravet@etu.univ-amu.fr

Daniel Overholt
CREATE, Aalborg University
Denmark
dano@create.aau.dk

Abstract

REMORA turns a martial arts staff (bô) into a musical instrument: spins, tilts, and strikes become sound in real time. A small wireless sensor unit on the staff tracks its motion and sends it to a companion audio plugin, which we built around a simple idea - instead of hard-wiring one fixed gesture-to-sound behaviour, the mapping itself is a small patch of connected building blocks that the performer can see and rewire live, on a visual canvas inside the plugin, much like a modular synthesiser. We show that this same small set of building blocks is enough to reproduce, as editable patches, everything from simple tilt-controlled pitch to compound movement-quality mappings inspired by Laban Movement Analysis. This paper describes the instrument's hardware, its self-calibration method, the mathematics behind turning raw orientation into musical control, and the live patching interface itself, and reflects on what it means to make an instrument's gestural vocabulary an explicit, editable part of the instrument.

CCS Concepts

• **Applied computing** → **Sound and music computing**.

Keywords

digital musical instrument, gestural control, inertial sensing, Laban Movement Analysis, OSC, staff instrument

1 Introduction

Motivation and Background

The bô staff is a full-body controller form factor that's had surprisingly little attention in NIME - most inertial-sensor DMIs mount their sensor on the body or a small handheld object, not a swung, two-handed staff with its own rotational inertia and reach. That gap is my opening case. The claim I want on the table early: REMORA's contribution isn't one more fixed gesture-to-sound mapping, but a reusable, block-rate visual programming layer embedded inside the instrument itself - every named mapping described later in the paper is really just one saved patch of that layer.

Objectives of the Study

Four things here feel genuinely new to me, and I still need to turn this list into proper prose: (1) a small dataflow language (source / math / sink node primitives) embedded directly in the audio plugin, evaluated as a DAG (Directed Acyclic Graph) once per audio block, that a performer can rewire live instead of me recompiling C++; (2) by construction, this same handful of primitives turns out to be expressive enough to reproduce eleven previously hand-coded mapping strategies as ordinary presets - the mapping design space I explored by hand over the course of this project is, it turns out, spanned by a fairly compact algebra; (3) a magnetometer-free world-frame heading estimate ("facing") derived purely from the calibrated orientation stream, which sidesteps the magnetic interference a stage rig would otherwise cause; (4) a mount-agnostic calibration transform, so the same instrument logic runs no matter where the sensor physically sits on the staff.

Scope and Applications

REMORA targets solo live performance with a single staff and a single IMU - not an installation piece, not a multi-performer controller (at least not yet, see Future Improvements). Worth being upfront about what's out of scope for this paper too: no formal user study, no controlled comparison against other staff instruments, and no claim that patching a mapping is easier to learn than a fixed one - only that the capability exists and is expressive.

2 Related Work

Staff, Rod, and Baton DMIs

I need to place REMORA against prior staff/rod-shaped DMIs - the T-Stick, conducting batons, poi/staff-flow instruments - covering how each senses motion and, more importantly, what vocabulary of gesture-to-sound mapping each exposes to the performer, since that vocabulary is the axis REMORA differs on most.

Inertial Sensing and Visual Patching in NIME

Two threads to cover here: prior IMU-based DMIs and how they handle calibration/orientation extraction, and - separately - visual dataflow patching environments (Max/MSP, Pure Data, VCV Rack) as the closest existing point of comparison for the node-graph interface. I should also fold in prior use of Laban Effort qualities as a mapping source in NIME work.

3 Design and System Architecture

Initial Design Requirements

Modelled on how a proper requirements list should read, not just a feature dump:

- *Full-body, swingable staff form factor - not a handheld controller*
- *Wireless sensing during performance; a cable only for charging*
- *Live-editable mapping, with no audio interruption when switching or rewiring*
- *Mount-agnostic calibration, independent of the sensor's physical placement on the staff*

- *Low enough end-to-end latency for the staff to feel driven, not laggy*

Architecture Overview

Here's where I lay out the two-part architecture: firmware-side sensing and OSC (Open Sound Control) transport on one end, plugin-side reception, calibration, graph evaluation, and synthesis on the other. The signal-flow diagram belongs in this section.

4 Implementation

The Build

This is where I go through the physical build: the staff-mounted enclosure, the parametric OpenSCAD casing, and how the electronics are secured to the staff without changing how it's normally gripped or swung.

Hardware

This is where I go through the ESP32-S2 + MPU-9250 sensor package, the 100 Hz I2C (Inter-Integrated Circuit) sampling loop, and the Wi-Fi/UDP (User Datagram Protocol)/OSC packet format the firmware streams to the host.

Software: Calibration and Motion Mathematics

This is the technical core of the paper. Every transformation below is transcribed from the actual implementation, not idealised - each item is tagged with the function(s) it lives in so the mapping from equation to code is traceable.

Orientation and Calibration. **Rotation.** All spatial reasoning works on unit quaternions rather than Euler angles, to avoid gimbal lock:

$$v' = q\, v\, q^{-1}$$

where v is the vector being rotated, q the unit orientation quaternion, and q^{-1} its conjugate. Computed via the standard expanded (non-multiplication) formula; Euler pitch/roll/yaw are only reconstructed once, at the very end, for the handful of mappings that still want them. `MathHelpers::rotate`, `rotateVectorByQuaternion`, `toEuler`, `multiply`, `conjugate`

Calibration, stage 1 - local axis selection. The three calibration poses q_A, q_B, q_C determine *which* body axis of the sensor is the staff's forward direction. For each of the six axis-aligned candidates $L \in \{\pm X, \pm Y, \pm Z\}$:

$$v_P = \widehat{\text{rotate}(L, q_P)}\,, \quad P \in \{A, B, C\} \qquad e(L) = |v_A \cdot v_B| + |v_B \cdot v_C| + |v_C \cdot v_A|$$

where L is the candidate body axis under test, q_P the quaternion recorded for pose P, and v_A, v_B, v_C the resulting normalised directions for the three poses. The candidate minimising $e(L)$, subject to the right-handedness constraint $(v_A \times v_B) \cdot v_C < 0$, is kept as the alignment quaternion q_{align}. `computeAlignQuat` (PluginProcessor.cpp)

Calibration, stage 2 - frame alignment. The chosen staff axis, rotated through each pose, gives three measured directions b_0, b_1, b_2 (forward/up/right), compared against the target directions r_0, r_1, r_2 of the virtual instrument frame. Both triads are orthonormalised by Gram-Schmidt:

$$e_0 = b_0, \qquad e_1 = \widehat{b_0 \times b_1}, \qquad e_2 = e_0 \times e_1$$
$$f_0 = r_0, \qquad f_1 = \widehat{r_0 \times r_1}, \qquad f_2 = f_0 \times f_1$$

and the correction rotation is the closed form

$$R = FE^{\top}\,, \qquad F = [\, f_0 \;\; f_1 \;\; f_2 \,]\,, \quad E = [\, e_0 \;\; e_1 \;\; e_2 \,]$$

the least-squares-optimal rotation aligning the measured triad to the target triad - the same result an orthogonal-Procrustes/SVD solve would give here, obtained without an SVD. R is converted back to a quaternion q_{corr} by Shepperd's method. `computeCorrection`, `MathHelpers::fromMatrix`

Calibration, runtime application. Every subsequent raw reading is expressed in instrument space as

$$q' = q_{\text{corr}}\,(q_{\text{raw}}\, q_{\text{align}})$$

where q_{raw} is the live quaternion reported by the sensor and q_{align}, q_{corr} the two calibration quaternions found above - independent of how the sensor is physically mounted on the staff. `REMORAProcessor::getCalibra`

Motion Feature Extraction. All of the following run once per received packet inside the shared analyzer every mapping graph reads from, so no mapping duplicates this logic. `StaffMotionAnalyzer::computeFrame` (StaffMotionAnalyzer.cpp/.h).

Timestep. Δt is measured between packets and clamped to (0.1ms, 200ms) so a dropped packet cannot spike any integrator. `calculateDeltaTime`

Tip vector and rotation axis. The staff-tip direction $p_n = \text{rotate}((1,0,0), q_n)$ is kept in a 3-sample ring buffer; the instantaneous rotation axis at the midpoint sample is obtained by central difference:

$$\bar{v} = \tfrac{1}{2}(p_0 - p_2) \qquad \text{axis} = p_1 \times \bar{v}$$

where subscripts 0, 1, 2 denote the newest, middle, and oldest of the three buffered tip vectors, and $\bar{v}$ is the estimated tip velocity at the middle sample. `updateTipPositionHistory`, `calculateRotationAxisAtMidpoint`

Gravity and dynamic acceleration. A gravity estimate is tracked as a smoothed inverse-rotation of the world $+Z$ axis,

$$g_n = \text{onePole}\big(g_{n-1},\ \text{rotate}(+Z,\ q_n^{-1}),\ \alpha{=}0.10\big)$$

then subtracted from the raw accelerometer reading to isolate motion from posture: $a_{\text{dyn}} = a_{\text{raw}} - g$, where a_{raw} is the raw accelerometer sample (in g) and g the smoothed gravity estimate g_n above. `updateGravityVector`, `calculateDynamicAcceleration`

One-pole smoothing. The single primitive reused by nearly every feature below:

$$y_n = y_{n-1} + \alpha\,(x_n - y_{n-1})$$

where x_n is the new input sample, y_{n-1} the previous smoothed output, and $\alpha \in (0, 1]$ the smoothing coefficient (higher = faster response). `MathHelpers::applyOnePoleFilter`

Laban Weight. Velocity is integrated from dynamic acceleration with exponential decay derived from an 80 ms half-life:

$$v_n = \big(v_{n-1} + a_{\text{dyn}} \cdot 9.81 \cdot \Delta t\big) \cdot e^{-\Delta t \cdot 1000 \ln 2/80}$$

where v_n is the running velocity estimate, 9.81 m/s^2 converts a_{dyn} from g-units to physical units, and the exponential factor discards accumulated velocity with an $\approx$ 80 ms half-life. Weight is $\|v_n\|$,

scaled and clamped, run through an asymmetric attack/release envelope (α_{attack}=0.40, α_{release}=0.006) so it snaps up on impact and settles slowly. `integrateVelocityForLabanWeight`, `updateLabanWeight`

Laban Time (suddenness). The derivative of smoothed gyroscope magnitude,

$$\|\dot{\omega}\| = \frac{\|\omega\|_n - \|\omega\|_{n-1}}{\Delta t}$$

where ω is the gyroscope's angular-velocity vector (in °/s) and $\|\omega\|_n$ its smoothed magnitude at block n. The result is clamped to its positive part, normalised by 800°/s², and given its own fast-attack/slow-release envelope. `updateLabanTime`

Laban Space (focus). The running absolute dot product of consecutive normalised gyroscope-axis directions, one-pole smoothed (α=0.10) and relaxing toward 0.5 whenever the staff is nearly still. `updateLabanSpace`

Laban Flow. Normalised jerk of the dynamic-acceleration magnitude,

$$j_{\text{flow}} = \frac{\left|\|\dot{a}_{\text{dyn}}\|\right|}{50}$$

where $\|\dot{a}_{\text{dyn}}\|$ is the block-to-block change in dynamic-acceleration magnitude. j_{flow} is clamped to $[0, 1]$ and smoothed (α=0.12) into `flow_bound`; `flow_free` = 1 − `flow_bound`. `updateLabanFlow`

Spin classification. The smoothed rotation axis (α=0.35) is classified vertical iff its horizontal-plane magnitude exceeds $|axis_z|$; spin direction is the sign of the dominant in-plane axis component in one convention, or the sign of that axis's dot product with a reference azimuth latched at gesture onset in the other. `updateSpinClassificationByAbsoluteComponent` (Azimut/Ben's), `...ByReferenceAzimuth` (Bozendo)

Facing. Deliberately not an atan2/compass-heading computation - a hysteresis comparison

$$|axis_x| \;\gtrless\; 1.15 \cdot |axis_y|$$

where $axis_x, axis_y$ are the horizontal components of whichever axis is currently in play - the smoothed rotation axis during vertical spins, or the smoothed tip direction during horizontal ones - giving ≈ 48°/42° asymmetric switch points so the classification does not chatter at the boundary. No magnetometer is involved. `updateFacingClassification`

Continuous spin count. Degrees are accumulated as $\|\omega\| \cdot \Delta t$; every 360° crossed decrements the accumulator and increments a signed lap counter, which resets to 0 the instant the spin classification changes. `accumulateContinuousSpins`

Axial thrust detection. Dynamic acceleration is rotated into world frame and projected onto the tip direction by dot product to give a signed axial acceleration a_{ax}; its jerk

$$j_{\text{ax}} = \frac{a_{\text{ax},n} - a_{\text{ax},n-1}}{\Delta t}$$

where $a_{\text{ax},n}$ and $a_{\text{ax},n-1}$ are the current and previous axial-acceleration samples. j_{ax} is watched for a zero-crossing coincident with $|a_{\text{ax}}| >$ 1.5g, gated to $\|\omega\| < 90°$/s (so a fast spin cannot be mistaken for a stab) and a 200 ms cooldown. A valid peak sets the thrust envelope to clamp(strength + 0.5, 0, 1), decaying ×0.94 per block (≈ 150 ms half-life). `detectAxialThrustPeaks`

Software: Mapping Architecture

Here I explain the `NodeGraph/NodeTypeRegistry` *design: three node categories (Source - raw IMU channels and derived motion/Laban features; Math - arithmetic, shaping, dynamics, lookup tables; Sink - synth parameters, from scalars like root frequency to indexed arrays like per-voice gain/pan), each a small typed function registered once. A graph is just nodes plus typed input-to-output wiring, serialised to/from XML (Extensible Markup Language) as a preset - and I want to be clear that* `GraphMappingStrategy` *makes the graph itself the only mapping type the synth ever talks to; there's no separate hand-written mapping path anymore.*

Graph Evaluation and Primitive Library. **DAG evaluation.** A patch is a directed graph of typed nodes; `NodeGraph::connect` provisionally adds an edge and recomputes a topological order with Kahn's algorithm (in-degree queue) via `NodeGraph::recomputeTopoOrder` - if the graph does not fully drain (a node's in-degree never reaches zero), the edge closed a cycle and is reverted. Each audio block, `NodeGraph::evaluate` walks the cached order once, and every node computes

$$y_i = f_{\text{type}(n_i)}\left(x_{i,1}, \ldots, x_{i,m}\,;\,\theta_i\right)$$

where n_i is the i-th node in topological order, type(n_i) its registered node-type function, $x_{i,1}, \ldots, x_{i,m}$ its wired input values, θ_i its parameters, and y_i its resulting output(s) - so the whole patch is one composed function of the raw sensor frame, evaluated once per block with no re-sorting on the audio thread.

Primitive vocabulary. Every node type is a single small pure function (or, for the few stateful ones, a function plus one scratch struct), registered once: arithmetic (add/subtract/multiply/divide/abs/max/min/equ shaping (mapRange, clamp, threshold, crossfade, quantizeSteps, sine, atan2, semitonesToHz), lookup tables (a general 1-D table and a 3-boolean-selector table - the same node type that implements the Azimut root table below), and stateful dynamics (variable-rate one-pole smoother, leaky integrator, retrigger envelope, hysteresis step, derivative, latched smoother). `Graph::buildAllNodes` (NodeMetadata.cpp)

Software: Named Mappings as Graph Presets

I'd rather show the generalisation claim than just assert it, so this subsection walks through how Simple (tilt-to-pitch), Bowed Chord, Lead+Drone, Spin Filter, the Bozendo V1/V2 Laban-driven mappings, and the Azimut family (facing-based eight-way root table, spin-count filter sweep, axial thrust transients, plus the Azimut+/Reverb/Ben's/Spin Voices variants) are each just a specific wiring of the same node vocabulary - built once in C++ via `GraphBuilder`*, and now editable as ordinary patches.*

Azimut-Specific Transformations. `Graph::Presets::buildAzimutCore` (AzimutCore.cpp) - shared by every Azimut-family preset.

Root-note table. The spin-plane/direction/facing root-note table above is not an if/else chain - it is literally an 8-entry lookup table indexed by three booleans (`isVertical`, `isCCW`, `isFacingEast`) through the graph's generic 3-selector LUT node.

Root morph. The target semitone value is chased by a variable-rate one-pole filter whose rate is

$$\text{rate} = \max\left(\text{clamp}\left(\tfrac{\|\omega\|}{2000},\ 0.005,\ 0.2\right),\ \ 0.5 \cdot \text{onsetEnv}\right)$$

where `onsetEnv` is a retrigger envelope (decay 0.90) that fires the instant the staff leaves rest - faster motion, or a fresh onset, morphs the root pitch in quicker.

Filter sweep. The spin count drives a phase modulator whose sine output is remapped onto the filter's cutoff range and smoothed:

$$\text{cutoff} = \text{onePole}\Big(\text{mapRange}\big(\sin(1.5\cdot\text{spinCount}),\ [-1,1],\ [400, 20000]\text{Hz}\big),\ \text{rate}{=}0.03\Big)$$

where spinCount is the signed lap counter from Continuous Spin Count above, mapRange linearly rescales its argument from the first interval to the second, and onePole is the smoothing primitive defined earlier. `buildSpinCountLpfHz`

Graphical Interface: Live Patching as Performance Practice

This is probably the subsection I want to spend the most space on after the mathematics. I want to present the node-graph editor as the instrument's primary authoring surface, not a debug tool: a pannable, zoomable canvas on which each node renders as a coloured card - green for Source, yellow for Sink, neutral for Math - labelled with its category and inline, live-editable parameter fields; typed input/output pins along each card's edges accept click-and-drag connections rendered as cubic-Bézier wires, colour-highlighted while dragging and hit-testable afterward for rewiring or deletion via context menu. New nodes are added from a categorised picker at any canvas position; an auto-layout pass untangles a freshly loaded preset into left-to-right source-to-sink columns. A dirty-state indicator tracks whether the active patch has diverged from its saved preset, with one-click revert - so a performer can improvise a mapping change mid-rehearsal and either keep or discard it, treating the mapping itself as something authored and iterated on at the same level as the sound it produces.

Software: Synthesis Engine

Still need to write up `BoStaffSynth` *here - the shared four-voice additive engine and its mapping-agnostic parameter surface (gain, drive, noise, filter, pan, reverb send) that every graph's sink nodes write into.*

Additive Synthesis Engine. `BoStaffSynth::processBlock`, `SynthVoice::tick` (BoStaffSynth.h/.cpp).

Harmonic bank. Each of the 4 voices sums 6 phase-accumulated sine partials at integer multiples of a (vibrato- and chorus-modulated) fundamental, each with its own independently smoothed target amplitude.

Waveshaping. A rational soft-clip saturator, applied per-voice when `driveAmt` > 0:

$$y = \frac{x\,(1 + \text{driveAmt})}{1 + \left|x\,(1 + \text{driveAmt})\right|}$$

where x is the voice's summed harmonic-bank sample before shaping and `driveAmt` the mapping-supplied drive amount.

Resonator. A short feedback comb (1800-sample delay, 0.35 feedback, mixed 0.7 dry / 0.3 wet) is layered onto every voice for a string-like coloration.

Modulation. Vibrato and tremolo are sinusoidal multipliers on pitch and gain respectively:

$$f' = f\,\big(1{+}\text{depth}{\cdot}\sin\phi_v\big) \qquad g' = g\,\big(1{-}\text{depth}{\cdot}(0.5{+}0.5\sin\phi_t)\big)$$

where f, g are the fundamental frequency and master gain before modulation, f', g' the modulated values actually used, and ϕ_v, ϕ_t the vibrato and tremolo LFO phase accumulators (radians), each advancing by its own rate every sample. A per-voice phase-offset sine (depth 0.003, rate 0.3 Hz) adds chorus detuning; chord changes cross-fade linearly between old and new semitone offsets over 200 ms rather than jumping.

Noise and filtering. An xorshift32 PRNG feeds a one-pole lowpass (coefficient `noiseLpCoef`) before being mixed in at `noiseAmount`; the summed stereo output passes through a JUCE state-variable TPT lowpass (cutoff from the graph's `sink.lpfCutoffHz`) and an optional Schroeder-style `juce::dsp::Reverb` send, both smoothed to avoid zippering when the active mapping changes.

Pitch/frequency conversion.

$$f = f_{\text{root}} \cdot 2^{\,\text{semitones}/12}$$

where f_{root} is the mapping's current root frequency in Hz and semitones the signed offset applied to it, giving the equal-tempered result f. `MathHelpers::semitoneRatio`, `convertSemitonesToHertz`

5 Development Iterations

Modelled on the reference paper's iteration log rather than presenting the final architecture as if it arrived fully formed. Mine reads roughly: (1) firmware + OSC transport + a single hardcoded tilt-to-pitch mapping, just to prove the sensor-to-sound pipeline worked at all; (2) hand-building the full progression of named mappings - Bowed Chord through Bozendo to Azimut and its variants - each one a bespoke C++ class; (3) noticing the hand-built mappings were all reducible to the same handful of primitives, and generalising that into the node-graph engine so every prior mapping became a preset instead of a class.

Iteration 1: Sensing Pipeline

Firmware reading the IMU and streaming OSC, a minimal receiver on the plugin side, and one hardcoded tilt-to-pitch mapping - the goal here was purely proving the pipeline end to end, not expressivity.

Iteration 2: Hand-Built Mapping Library

The eleven named mappings built one at a time as their own C++ classes - Simple, Bowed Chord, Lead+Drone, Spin Filter, Bozendo V1/V2, Azimut and its variants, Ben's, Spin Voices - each adding a new gesture vocabulary (chords, Laban Effort, world-frame facing, transient detection) on top of the shared `StaffMotionAnalyzer`.

Iteration 3: Generalising into the Node Graph

Recognising that every hand-built mapping was really just arithmetic, shaping, and a few stateful primitives wired together in a fixed pattern - and rebuilding the whole mapping layer as a small generic graph engine, with the eleven prior mappings ported over as presets rather than deleted.

6 Evaluation

No formal study yet - I want to be honest about that rather than dress up informal playtesting as more than it is.

Test Session with a Performer

TBD - modelled on the reference paper's semi-structured test session: get someone who actually performs with staff-like objects (martial arts, poi, staff spinning) to try REMORA, and take notes rather than assume my own playtesting generalises.

Notes and Suggestions. TBD once the session happens.

In Summary. TBD once the session happens.

7 Discussion

Advantages and Limitations

I want to be honest here about what the generalisation into a graph actually buys versus what it costs: expressiveness and shareability of mappings, weighed against the added cognitive step of patching for performers used to fixed instruments. Worth folding in the informal playtesting observations, and I should be upfront about the current limitations - single-IMU ambiguity, no formal user study yet, no perceptual evaluation of the facing/calibration accuracy.

Future Improvements

Extensions I keep coming back to: gesture-triggered mode switching, a merged rest/motion mapping, a two-IMU dual-tip configuration, further Laban Effort mapping targets, ML-based discrete gesture recognition, and eventually exposing the node graph itself as a shareable/community patch format.

8 Conclusions

I want to close by making REMORA's real novelty explicit: it's architectural, not just one more instrument - collapsing eleven hand-built gesture mappings into presets of one small, live-patchable dataflow language embedded in the plugin, grounded in an explicit mathematical account of calibration, orientation, and movement-quality extraction.

Acknowledgments

This work was carried out during the first author's internship at CREATE, Aalborg University, Denmark.

9 Ethical Statement

Needs real content before submission, but the shape should cover: accessibility (the staff form factor assumes a certain range of mobility and two-handed grip strength - name that as a limitation rather than leaving it unstated); environmental impact of the staff-mounted hardware build (battery, 3D-printed/laser-cut enclosure materials, whether they're recyclable); socio-economic fairness (REMORA is built entirely on FLOSS/FLOSH tools - JUCE, PlatformIO, OpenSCAD - worth stating plainly, since that's a genuine point in its favour under the NIME Code of Practice); no human-participant data collected beyond the informal test session in the Evaluation section, with informed consent obtained for that session; and disclosure of any AI assistance used in drafting this manuscript, per the NIME Code of Practice on Ethical Research - separate from the instrument's code, which was written entirely by hand.